



Python Basics for Data Science

Conditionals and Loops - Nền tảng lập trình Python

Quanskill

Introduction

Python Conditionals

Python Loops

Best Practices

Exercises

Introduction

- **Decision Making:** Xử lý dữ liệu dựa trên điều kiện
- **Data Processing:** Lặp qua từng dòng, cột của dataset
- **Model Training:** Kiểm soát quá trình huấn luyện mô hình
- **Data Validation:** Kiểm tra tính hợp lệ của dữ liệu

Ví dụ thực tế

Phân loại khách hàng theo tuổi, xử lý missing values, training epochs trong machine learning...

Trước khi bắt đầu, đảm bảo bạn có:

- Python 3.x đã cài đặt
- IDE/Text Editor (VS Code, PyCharm, Jupyter Notebook)
- Hiểu biết cơ bản về variables và data types

```
1 # Test your Python environment
2 print("Hello, Data Science!")
3 # Output: Hello, Data Science!
```

Python Conditionals

Khái niệm: Cho phép chương trình thực hiện các hành động khác nhau dựa trên điều kiện.

Syntax cơ bản:

```
1 if condition:
2     # Execute this code
3 elif another_condition:
4     # Execute this code
5 else:
6     # Execute this code
```

Lưu ý quan trọng

Python sử dụng **indentation** (thụt lề) để xác định code block thay vì dấu ngoặc nhọn {}

```
1 # Example 1: Simple age check
2 age = 18
3
4 if age >= 18:
5     print("You are an adult")
6 # Output: You are an adult
7
8 # Example 2: Data Science context
9 data_accuracy = 0.95
10
11 if data_accuracy > 0.9:
12     print("Model performance is excellent!")
13 # Output: Model performance is excellent!
```

Giải thích:

- Điều kiện `age >= 18` được kiểm tra
- Nếu True, code bên trong được thực thi
- Indentation (4 spaces) bắt buộc


```
1 # Example: Customer classification
2 purchase_amount = 250
3
4 if purchase_amount >= 500:
5     customer_type = "Premium"
6     discount = 0.2
7 else:
8     customer_type = "Regular"
9     discount = 0.1
10
11 print(f"Customer type: {customer_type}")
12 print(f"Discount: {discount * 100}%")
13
14 # Output:
15 # Customer type: Regular
16 # Discount: 10.0%
```

Phân tích:

- Nếu `purchase_amount >= 500`: khách hàng Premium
- Ngược lại: khách hàng Regular
- Chỉ một trong hai block được thực thi

```
1  # Example: Grade classification for student performance
2  score = 85
3
4  if score >= 90:
5      grade = "A"
6      performance = "Excellent"
7  elif score >= 80:
8      grade = "B"
9      performance = "Good"
10 elif score >= 70:
11     grade = "C"
12     performance = "Average"
13 else:
14     grade = "F"
15     performance = "Needs Improvement"
16
17 print(f"Score: {score}, Grade: {grade}")
18 print(f"Performance: {performance}")
19
20 # Output:
21 # Score: 85, Grade: B
22 # Performance: Good
```

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

```
1 # Examples with output
2 print(5 == 5)      # Output: True
3 print(5 != 3)      # Output: True
4 print(10 > 5)       # Output: True
5 print(3 <= 3)       # Output: True
6
7 # Data Science example
8 accuracy = 0.85
9 print(accuracy >= 0.8) # Output: True
```

Operator	Description
and	Both conditions must be True
or	At least one condition must be True
not	Inverts the boolean value

```
1 # Example: Data validation
2 age = 25
3 income = 50000
4
5 # Using 'and'
6 if age >= 18 and income >= 30000:
7     print("Eligible for premium credit card")
8 # Output: Eligible for premium credit card
9
10 # Using 'or'
11 if age < 18 or income < 20000:
12     print("Basic account recommended")
13 else:
14     print("Standard account available")
15 # Output: Standard account available
```

```
1 # Example: ML model evaluation
2 model_accuracy = 0.92
3 dataset_size = 1500
4
5 if model_accuracy > 0.9:
6     print("High accuracy model detected")
7
8     if dataset_size > 1000:
9         print("Large dataset - Model is reliable")
10        recommendation = "Deploy to production"
11    else:
12        print("Small dataset - Need more data")
13        recommendation = "Collect more training data"
14 else:
15     recommendation = "Improve model performance"
16
17 print(f"Recommendation: {recommendation}")
18
19 # Output:
20 # High accuracy model detected
21 # Large dataset - Model is reliable
22 # Recommendation: Deploy to production
```

```
1 # Example: Cleaning and validating data entries
2 data_entries = [25, -5, 150, 0, 85, None]
3
4 for entry in data_entries:
5     if entry is None:
6         print(f"Entry: {entry} -> Missing value detected")
7         status = "MISSING"
8     elif entry < 0:
9         print(f"Entry: {entry} -> Invalid negative value")
10        status = "INVALID"
11    elif entry > 100:
12        print(f"Entry: {entry} -> Outlier detected")
13        status = "OUTLIER"
14    else:
15        print(f"Entry: {entry} -> Valid data")
16        status = "VALID"
17
18 # Output:
19 # Entry: 25 -> Valid data
20 # Entry: -5 -> Invalid negative value
21 # Entry: 150 -> Outlier detected
22 # Entry: 0 -> Valid data
23 # Entry: 85 -> Valid data
24 # Entry: None -> Missing value detected
```

Python Loops

Khái niệm: Cho phép thực hiện một đoạn code nhiều lần mà không cần viết lặp lại.

Ứng dụng trong Data Science:

- Xử lý từng dòng dữ liệu trong dataset
- Training model qua nhiều epochs
- Tính toán statistics cho nhiều features
- Data preprocessing và transformation

Hai loại chính:

- **for loop:** Lặp qua sequence (list, string, range...)
- **while loop:** Lặp khi điều kiện còn đúng

Syntax:

```
1 for item in sequence:
2     # Execute this code for each item
```

Ví dụ đơn giản:

```
1 # Example 1: Iterate through a list
2 students = ["Alice", "Bob", "Charlie"]
3
4 for student in students:
5     print(f"Hello, {student}!")
6
7 # Output:
8 # Hello, Alice!
9 # Hello, Bob!
10 # Hello, Charlie!
11
12 # Example 2: Data Science context
13 scores = [85, 92, 78, 96, 88]
14
15 for score in scores:
16     print(f"Score: {score}")
17
18 # Output:
19 # Score: 85
20 # Score: 92
21 # Score: 78
22 # Score: 96
23 # Score: 88
```

For Loop với range()

```
1 # range() function creates sequence of numbers
2 print(list(range(5)))           # Output: [0, 1, 2, 3, 4]
3 print(list(range(1, 6)))        # Output: [1, 2, 3, 4, 5]
4 print(list(range(0, 10, 2)))    # Output: [0, 2, 4, 6, 8]
5
6 # Example: Processing data with indices
7 data = [23, 45, 67, 12, 89]
8
9 for i in range(len(data)):
10     print(f"Index {i}: Value {data[i]}")
11
12 # Output:
13 # Index 0: Value 23
14 # Index 1: Value 45
15 # Index 2: Value 67
16 # Index 3: Value 12
17 # Index 4: Value 89
18
19 # Calculate running average
20 total = 0
21 for i in range(len(data)):
22     total += data[i]
23     avg = total / (i + 1)
24     print(f"Running average after {i+1} items: {avg:.2f}")
```

```
1 # enumerate() gives both index and value
2 features = ["age", "income", "education", "experience"]
3
4 for index, feature in enumerate(features):
5     print(f"Feature {index + 1}: {feature}")
6
7 # Output:
8 # Feature 1: age
9 # Feature 2: income
10 # Feature 3: education
11 # Feature 4: experience
12
13 # Data Science example: Feature importance
14 feature_importance = [0.25, 0.35, 0.15, 0.25]
15
16 for i, (feature, importance) in enumerate(zip(features, feature_importance)):
17     print(f"Rank {i+1}: {feature} (importance: {importance})")
18
19 # Output:
20 # Rank 1: age (importance: 0.25)
21 # Rank 2: income (importance: 0.35)
22 # Rank 3: education (importance: 0.15)
23 # Rank 4: experience (importance: 0.25)
```

Syntax:

```
1 while condition:
2     # Execute this code while condition is True
3     # Remember to update condition variable!
```

Ví dụ cơ bản:

```
1 # Example 1: Simple counting
2 count = 0
3
4 while count < 5:
5     print(f"Count: {count}")
6     count += 1 # Important: update the variable!
7
8 # Output:
9 # Count: 0
10 # Count: 1
11 # Count: 2
12 # Count: 3
13 # Count: 4
14
15 print("Loop finished!")
16 # Output: Loop finished!
```

```
1 # Example: Model training simulation
2 epoch = 0
3 accuracy = 0.5
4 target_accuracy = 0.9
5
6 print("Starting model training...")
7
8 while accuracy < target_accuracy and epoch < 100:
9     # Simulate training (accuracy improves gradually)
10    epoch += 1
11    accuracy += 0.05 # Simulate improvement
12
13    print(f"Epoch {epoch}: Accuracy = {accuracy:.2f}")
14
15    # Early stopping condition
16    if accuracy >= target_accuracy:
17        print(f"Target accuracy reached at epoch {epoch}!")
18        break
19
20 # Output:
21 # Starting model training...
22 # Epoch 1: Accuracy = 0.55
23 # Epoch 2: Accuracy = 0.60
24 # ...
25 # Epoch 8: Accuracy = 0.90
26 # Target accuracy reached at epoch 8!
```

```
1 # break: Exit the loop immediately
2 numbers = [1, 2, 3, -1, 4, 5]
3
4 for num in numbers:
5     if num < 0:
6         print("Negative number found, stopping!")
7         break
8     print(f"Processing: {num}")
9
10 # Output:
11 # Processing: 1
12 # Processing: 2
13 # Processing: 3
14 # Negative number found, stopping!
15
16 # continue: Skip current iteration
17 scores = [85, -1, 92, 78, -1, 96]
18
19 for score in scores:
20     if score < 0:
21         print("Invalid score, skipping...")
22         continue
23     print(f"Valid score: {score}")
24
25 # Output:
26 # Valid score: 85
27 # Invalid score, skipping...
28 # Valid score: 92
29 # Valid score: 78
30 # Invalid score, skipping...
31 # Valid score: 96
```

```
1 # Example: Processing 2D data (matrix)
2 data_matrix = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 print("Matrix elements:")
9 for row_index, row in enumerate(data_matrix):
10     for col_index, value in enumerate(row):
11         print(f"Row {row_index}, Col {col_index}: {value}")
12
13 # Output:
14 # Matrix elements:
15 # Row 0, Col 0: 1
16 # Row 0, Col 1: 2
17 # Row 0, Col 2: 3
18 # Row 1, Col 0: 4
19 # Row 1, Col 1: 5
20 # Row 1, Col 2: 6
21 # Row 2, Col 0: 7
22 # Row 2, Col 1: 8
23 # Row 2, Col 2: 9
24
25 # Calculate row sums
26 for i, row in enumerate(data_matrix):
27     row_sum = sum(row)
28     print(f"Sum of row {i}: {row_sum}")
```

Best Practices

1. Code Readability:

```
1 # Good: Clear variable names
2 for student_score in test_scores:
3     if student_score >= passing_grade:
4         print("Pass")
5
6 # Avoid: Unclear variable names
7 for x in y:
8     if x >= z:
9         print("Pass")
```

2. Avoid Infinite Loops:

```
1 # Good: Always update loop variable
2 counter = 0
3 while counter < 10:
4     print(counter)
5     counter += 1 # Important!
6
7 # Bad: Infinite loop (missing update)
8 # while counter < 10:
9 #     print(counter) # counter never changes!
```

3. Use appropriate loop type:

```
1 # Use for loop when you know the number of iterations
2 data = [1, 2, 3, 4, 5]
3 for item in data: # Better than while loop here
4     print(item)
5
6 # Use while loop for condition-based iteration
7 accuracy = 0.5
8 while accuracy < 0.9: # Don't know exact iterations needed
9     accuracy += 0.1
```

4. Handle edge cases:

```
1 # Always check for empty data
2 data = []
3
4 if len(data) > 0: # Check before processing
5     for item in data:
6         print(item)
7 else:
8     print("No data to process")
```

1. Indentation Errors

```
1 # Wrong indentation
2 if x > 5:
3     print("Greater") # Error: No indentation
4
5 # Correct
6 if x > 5:
7     print("Greater") # 4 spaces indentation
```

2. Assignment vs Comparison

```
1 # Wrong: Assignment (=) instead of comparison (==)
2 if x = 5: # Error!
3     print("Equal")
4
5 # Correct
6 if x == 5: # Comparison
7     print("Equal")
```

Exercises

Exercise 1: Viết chương trình phân loại BMI

```
1 # BMI Categories:
2 # < 18.5: Underweight
3 # 18.5-24.9: Normal weight
4 # 25-29.9: Overweight
5 # >= 30: Obese
6
7 heights = [1.75, 1.65, 1.80, 1.70] # meters
8 weights = [70, 55, 85, 90]         # kg
9
10 # Your code here...
```

Exercise 2: Tìm số lớn nhất trong list

```
1 numbers = [23, 67, 45, 12, 89, 34, 56]
2
3 # Find maximum without using max() function
4 # Your code here...
```

Exercise 3: Đếm số từ trong một câu

```
1 sentence = "Python is great for data science and machine learning"
2
3 # Count words without using split() or len()
4 # Your code here...
```

Questions?